numba-cuda vs numba-cuda-mlir

Same 9 kernels, one source file, two compilers. Holding the GPU, CUDA 13.3 ptxas, workload and Python version constant — varying only the codegen frontend (NVVM vs MLIR). JIT time · PTX/SASS codegen · runtime & FPS · CUDA graphs.

bit-exact
OUTPUT EQUIVALENCE

≈ 1024²
LAUNCH→COMPUTE CROSSOVER

parity
UNDER CUDA GRAPHS

01 · METHOD — WHAT IS HELD CONSTANT

One variable: the compiler frontend

	CLASSIC	MLIR
package	numba-cuda 0.30.2	numba-cuda-mlir 0.4.0
frontend	Numba IR → LLVM → NVVM → PTX	MLIR → PTX (LTOIR)
opt / LTO default	NVVM opt-3, no LTO	LTO, opt>0
Python	3.12.13	3.12.13
cuda-bindings / ptxas	13.3.1 / CUDA 13.3	13.3.1 / CUDA 13.3
GPU	RTX 5880 Ada (sm_89), driver 595.71.05	

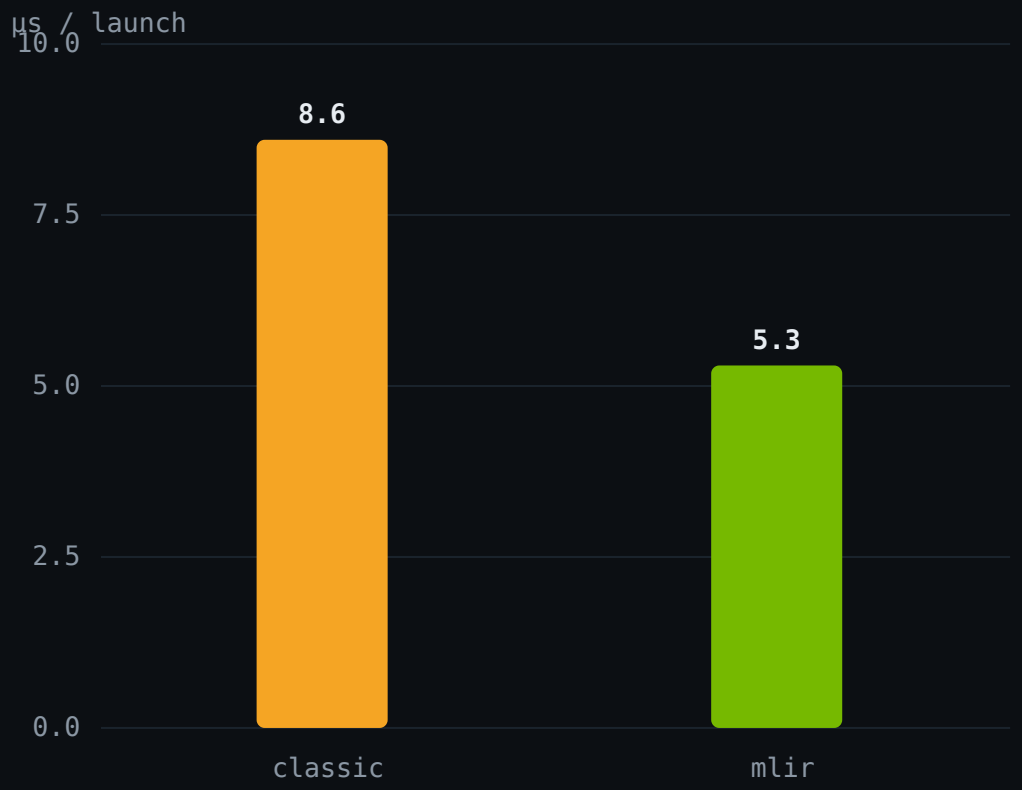
Bit-exact output on all 9 kernels at 256^2 / 512^2 / 1024^2 / 2048^2 → a fair race on identical work.

02 · TREND — FPS VS PROBLEM SIZE

The launch-bound → compute-bound crossover

Small grids = **host-launch-bound** (37 launches/frame): mlir wins **~38%**. Big grids = GPU-bound: **identical**.

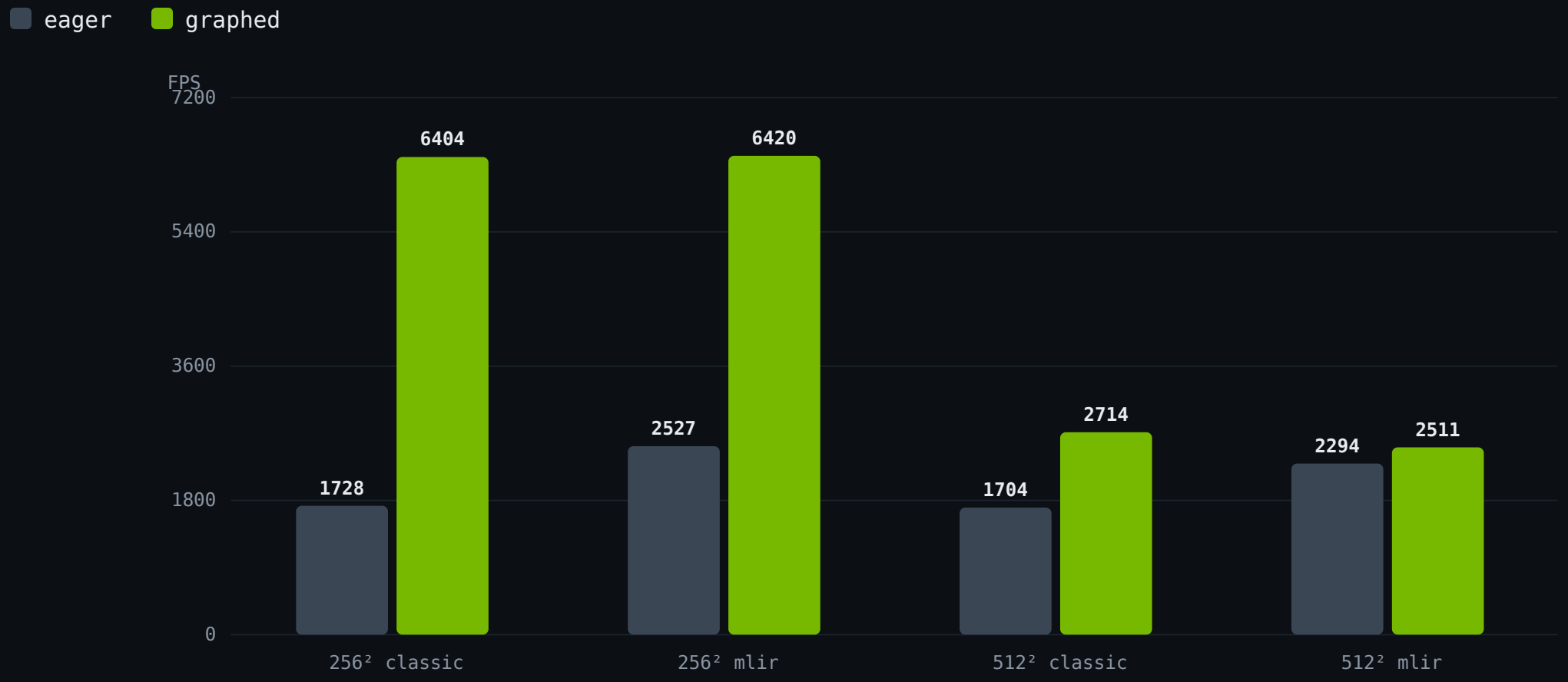
Equal GPU time; unequal dispatch cost



- GPU time/frame is **equal** — 0.34 ms both (nsys); 30 regs, 79% SM (ncu).
- The gap is pure dispatch: **8.6 µs** vs **5.3 µs** per launch.
- × 37 launches/frame = the **entire** difference.

04 · THE VECTOR THAT FLIPS THE HEADLINE

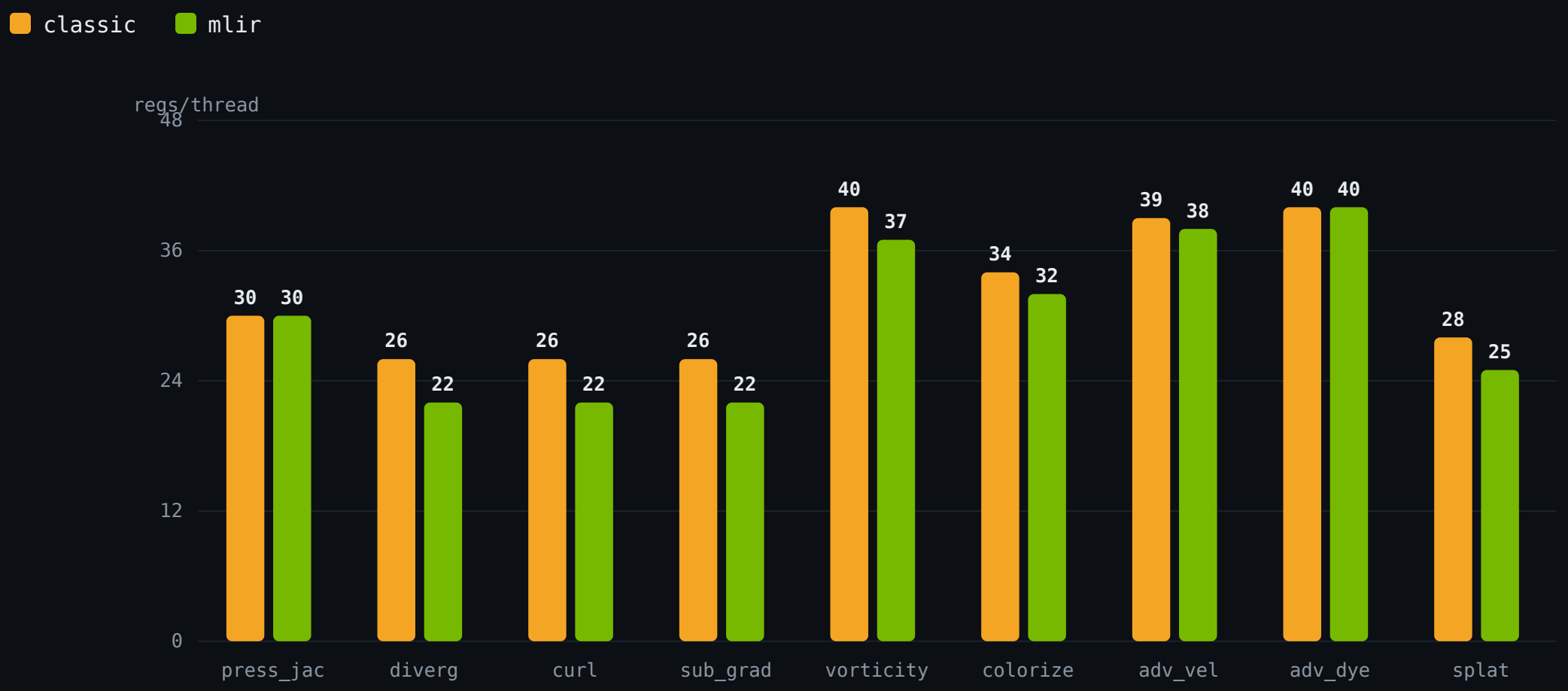
CUDA graphs erase the gap



Capture once, replay as a single launch. Both **converge to the same FPS** — classic gains more (**3.7×** vs 2.5×). mlir's eager edge **disappears**.

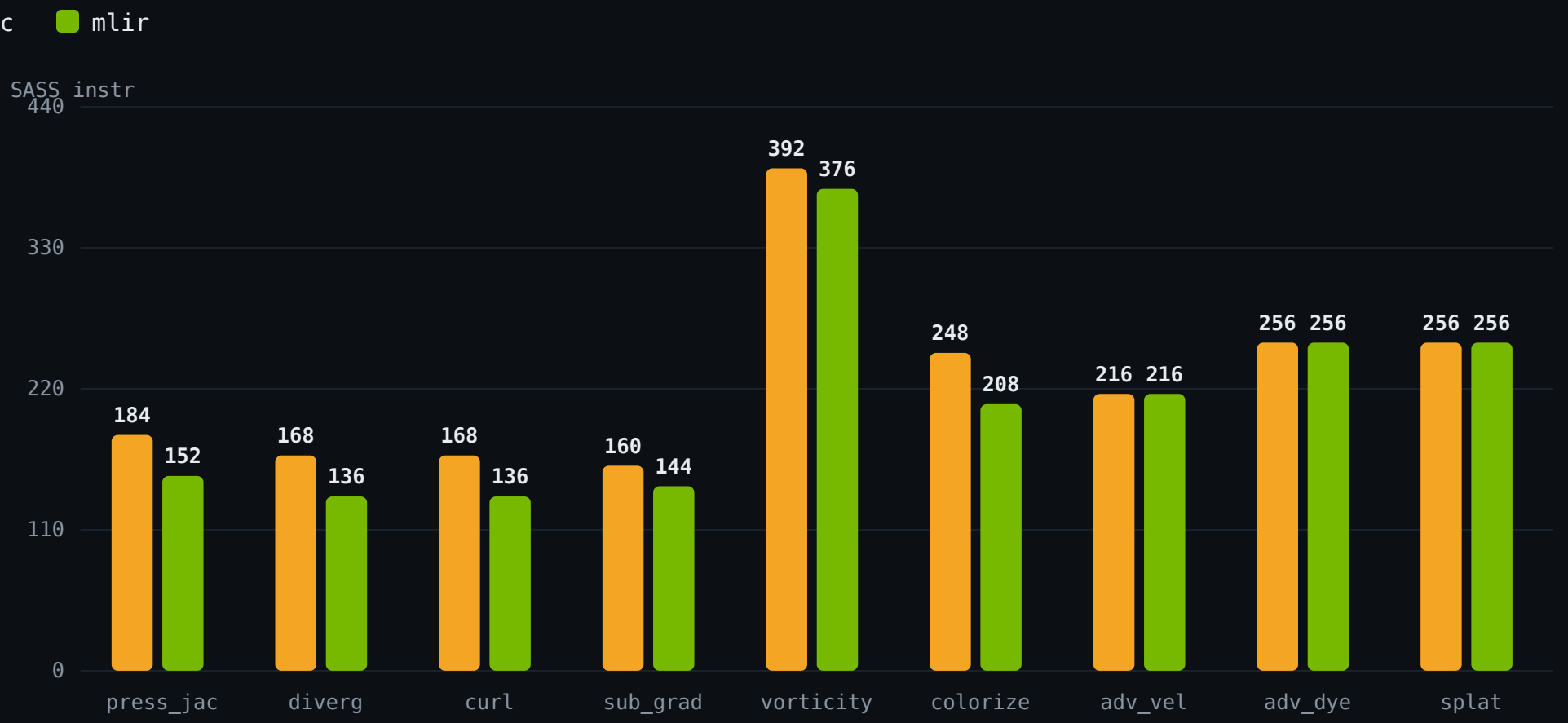
05 · CODEGEN — REGISTERS PER THREAD

Slightly fewer registers — and it doesn't matter



All below the 42-reg occupancy limit → **no** occupancy change. Dominant **pressure_jacobi**: 30 = 30.

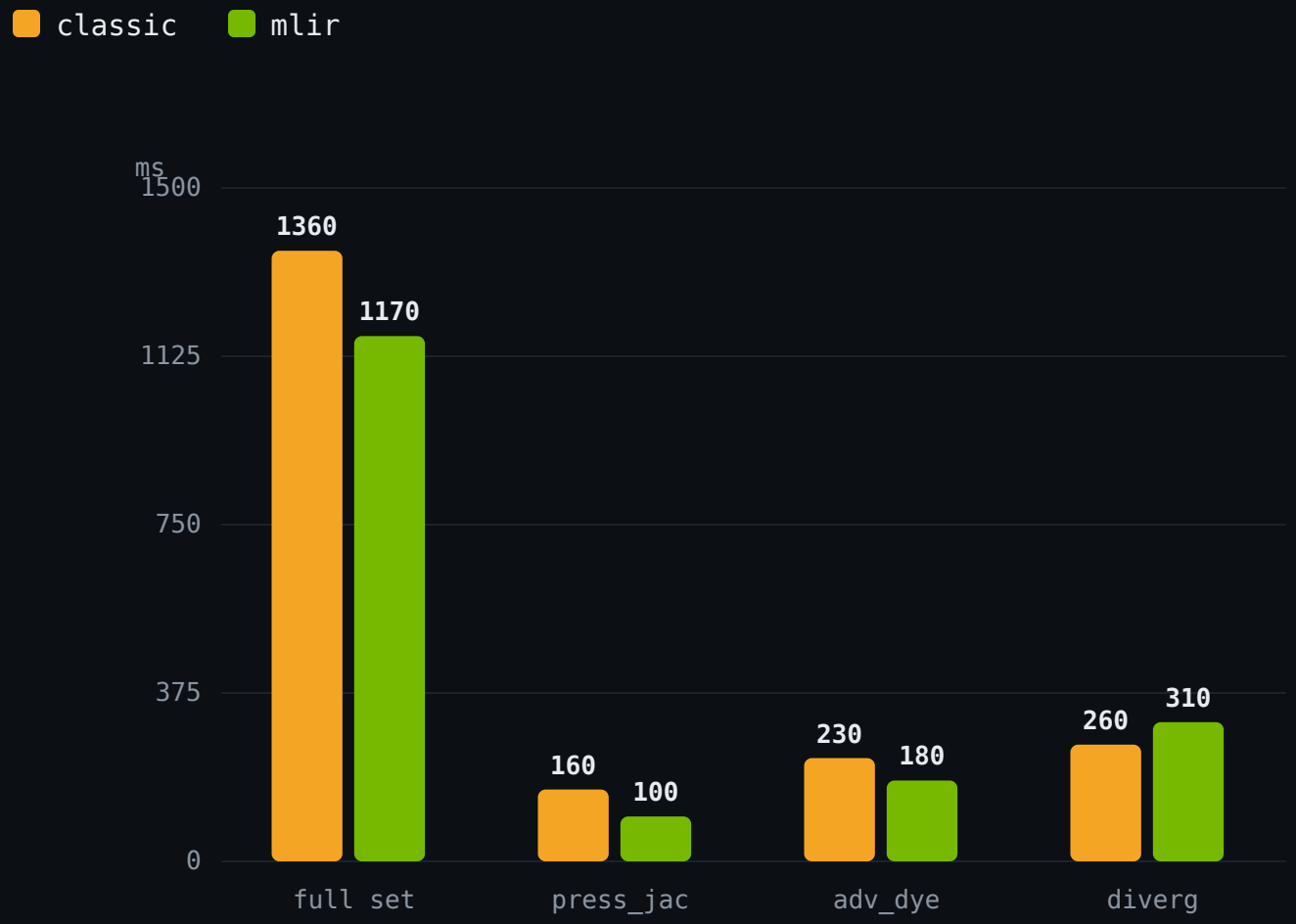
Fewer instructions ≠ faster here



mlir trims −10–19% on stencils, classic wins colorize — yet **GPU time is identical**. The limiter is **float64 throughput** (Python literals promote), not instruction count.

07 · JIT COMPILATION TIME

mlir compiles ~14% faster



1.36s
CLASSIC — FULL SET

1.17s
MLIR — FULL SET (3.12)

Context pre-warmed → pure compile. No on-disk cache, so paid every process.

The tightness is LTO, not the frontend

PRESSURE_JACOBI REGS	LTO OFF	LTO ON
classic (NVVM)	30	30
mlir (MLIR)	**50**	30

- At **matched LTO the frontends are identical** — 30 = 30.
- mlir's default-config edge is the **LTO pipeline**; the MLIR frontend alone is **worse** (50).
- classic's NVVM reaches 30 **without** LTO.

09 · VERDICT PER VECTOR

Same machine code; different launch path

VECTOR	WINNER	MAGNITUDE
equivalence	tie	bit-exact, all sizes
GPU codegen (runtime)	tie	equal GPU time / occupancy
static SASS (default)	mlir	−10–19% — but = at matched LTO
JIT compile time	mlir	~14% faster
eager FPS < 1024^2	mlir	+38% (5.3 vs 8.6 µs/launch)
FPS ≥ 1024^2 · with graphs	tie	compute-bound parity

Pick on launch model, not codegen

- Equivalent GPU code: **bit-exact**, equal runtime, equal occupancy.
- Only real gap: eager launch overhead (mlir **−38%**) — erased by CUDA graphs. mlir also compiles ~14% faster.
- Win #1: **float32 literals** kill the float64 promotion — both backends.
- Win #2: **CUDA graphs** ≈**3× FPS** when launch-bound — both backends.